SEMINAR S1-7 (CO5151 – ADVANCED AGENTIC AI)

Agent Design Patterns & Orchestration

Workflows vs. Autonomous Agents: Vì sao kiến trúc đơn giản nhất thường là lựa chọn đúng

Nhóm G4:
Đặng Lâm Tùng (Trưởng nhóm) · **Nguyễn Trung Phong** · **Vũ Việt Hưng**
Giảng viên hướng dẫn: **TS. Lê Xuân Bách** | Ngày báo cáo: 21/10/2026

Thời lượng: 30 phút · 23 slides

Phân Công & Thời Lượng Trình Bày

Mỗi thành viên phụ trách một phần, kết hợp lý thuyết với demo và số liệu thực nghiệm

Phần I: Workflows & Các Pattern Cơ Bản

Nguyễn Trung Phong

(9.5 phút | Slides 5–9)

- Workflows vs. Agents
- Vì sao Anthropic khuyên bắt đầu đơn giản
- Prompt Chaining & Routing
- Evaluator-Optimizer
- Khi nào mới cần đến agent

Phần II: Multi-Agent Orchestration

Vũ Việt Hưng

(9.5 phút | Slides 10–15)

- Orchestrator-Workers vs. long context
- Mô hình Orchestrator-Workers (Anthropic)
- Anthropic 2025: Lead/Subagent
- OpenAI 2025: tool loop, guardrails, evals
- LangGraph: cyclic state graph
- Durable execution & checkpointing

Phần III: Trade-offs & Fault Tolerance

Đặng Lâm Tùng (Trưởng nhóm)

(11 phút | Slides 16–22)

- Trade-off matrix
- Dynamic orchestration (G4)
- Pattern được chọn & bị loại
- Deterministic oracles
- State persistence & rollback
- Hub-and-spoke, swarm, graph
- Bài học production

Mục tiêu của buổi báo cáo

Trả lời đủ 6 câu hỏi trong đề cương, có code demo và số liệu đi kèm.

Kế Hoạch Nộp Bài & Các Mốc Thời Gian

Các mốc chính và 4 sản phẩm cần nộp

Các mốc thời gian

1. **Chủ Nhật, 18/10/2026 – 23:59**
   → **Hạn nộp bài** qua email.
   → Tiêu đề email: `[CO5151][G4] Seminar S1-7`.
2. **Thứ Hai & Thứ Ba (19–20/10)**
   → **Faculty audit window**: Thầy xem slide và chỉ ra những chỗ chưa chặt về kỹ thuật hoặc số liệu.
   → *Nộp trễ sẽ không nhận được pre-defense feedback.*
3. **Thứ Tư, 21/10/2026**
   → **Báo cáo chính thức trên lớp** (30 phút).

4 Sản Phẩm Cần Nộp

1. `slides_s1_7.pdf`
   → 23 slides.
2. `annotated_bibliography.pdf`
   → 1 trang: 1 core reading + 3 extension readings, mỗi bài kèm 1 câu critical reflection.
3. `demo_orchestration_trace.py` **& code**
   → Script đo token, latency và minh họa checkpoint recovery khi gặp văn bản luật đã bị bãi bỏ.
4. `ai_use_statement.pdf`
   → Bản khai báo việc sử dụng AI theo quy định của trường.

Tài Liệu Tham Khảo: Core & Extension Readings

1 core reading của Anthropic và 3 extension readings

Core Reading (bắt buộc cho cả lớp)

Anthropic (Dec 2024): *Building Effective Agents*

- Phân biệt **workflows** và **agents**.
- Các building block: prompt chaining, routing, parallelization, orchestrator-workers, evaluator-optimizer.
- **Luận điểm chính:** *"When building applications with LLMs, we recommend finding the simplest solution possible, and only increasing complexity when needed."*

Extension Readings

1. **Anthropic (Jun 2025)**: *How We Built Our Multi-Agent Research System*
   → Lead/subagent, context isolation; multi-agent tốn khoảng 15× token so với chat.
2. **OpenAI (Apr 2025)**: *A Practical Guide to Building Agents*
   → Tool-calling loop, guardrails, evals, khi nào nên tách thành nhiều agent.
3. **LangGraph Documentation (2024–2025)**
   → Cyclic state graph, durable execution, checkpointing, human-in-the-loop.

Câu Hỏi Trọng Tâm Của Seminar

Vì sao Anthropic khuyên chọn giải pháp đơn giản nhất, và khi nào thì chi phí token của multi-agent orchestration (khoảng 15× so với chat) là đáng bỏ ra?

Workflows vs. Agents: Ai Nắm Control Flow?

Thuyết trình: Nguyễn Trung Phong | Code path cố định hay để model tự quyết định

Workflows: Control Flow Nằm Trong Code

- **Control flow:** viết sẵn bằng code (Python, Go...).
- **Vai trò của LLM:** thực hiện từng bước nhỏ trong một pipeline đã định trước.
- **Routing:** if-else, kiểm tra schema, chuyển trạng thái theo quy tắc cố định.
- **Ưu điểm:** dễ dự đoán, dễ tái lập, latency thấp, ít token ($1\times$).

Agents: Model Tự Quyết Định Control Flow

- **Control flow:** do LLM quyết định trong vòng lặp (ReAct, tool calling).
- **Vai trò của LLM:** tự chọn tool, quyết định khi nào dừng và đi tiếp hướng nào.
- **Routing:** thay đổi theo observation ở runtime.
- **Đánh đổi:** linh hoạt với bài toán mở, nhưng lỗi dễ cộng dồn và chi phí khó kiểm soát.

Định Nghĩa Của Anthropic (2024)

"Workflows are systems where LLMs and tools are orchestrated through predefined code paths. Agents, on the other hand, are systems where LLMs dynamically direct their own processes and tool usage, maintaining control over how they accomplish tasks."

Vì Sao Anthropic Khuyên Ưu Tiên Workflow?

Thuyết trình: Nguyễn Trung Phong | Lỗi cộng dồn ($P = p^n$), chi phí token và rủi ro vận hành

Lỗi Cộng Dồn Theo Số Bước ($P = p^n$)

Giả sử mỗi bước đúng với xác suất $p = 0.95$ và các bước độc lập với nhau:

- **Workflow 3 bước:** $P_{\text{success}} = 0.95^3 \approx \mathbf{85.7}\%$
- **Agent loop 8 bước:** $P_{\text{success}} = 0.95^8 \approx \mathbf{66.3}\%$
- **Agent loop 15 bước:** $P_{\text{success}} = 0.95^{15} \approx \mathbf{46.3}\%$

Nhận xét: Độ tin cậy giảm theo hàm mũ khi trajectory dài ra. Workflow biết trước số bước và chèn được validation giữa các bước; agent thì không biết trước mình sẽ chạy bao nhiêu bước.

Bốn Rủi Ro Khi Trao Quyền Tự Chủ Quá Sớm

1. **Unbounded latency:** số turn do model quyết định nên thời gian phản hồi khó dự đoán.
2. **Token tăng nhanh:** history dồn lên sau mỗi turn, tổng token tăng gần $O(N^2)$.
3. **Khó debug:** hành vi non-deterministic, rất khó tái hiện lỗi ở edge case.
4. **Context drift:** tool output nhiễu tích tụ trong context, kéo theo hallucination dây chuyền.

Trả Lời Câu Hỏi Trong Đề Cương

Vì sao Anthropic khuyên không dùng agent cho phần lớn bài toán? Vì *"agentic systems often trade latency and cost for better task performance"*. Với task đã rõ các bước, workflow ổn định hơn, rẻ hơn và dễ debug hơn. Anthropic viết thẳng: *"This might mean not building agentic systems at all."*

Pattern Cơ Bản: Prompt Chaining và Routing

Thuyết trình: Nguyễn Trung Phong | Cách hoạt động, lợi ích và cái giá phải trả

Pattern 1: Prompt Chaining

Cơ chế: tách task thành một chuỗi LLM call nối tiếp; output của bước trước là input của bước sau.

What it buys:

- Mỗi prompt chỉ lo một việc, nên dễ viết và dễ kiểm soát.
- Chèn được validation (gate) bằng code giữa các bước.
- Các bước trung gian có thể dùng model nhỏ, rẻ hơn.

What it costs:

- Latency cộng dồn theo số bước ($T = \sum t_i$).
- Cứng nhắc: một bước sai mà không có code rẽ nhánh thì lỗi đi thẳng xuống các bước sau.

Pattern 2: Routing

Cơ chế: một classifier (LLM, regex hoặc embedding) xác định intent rồi chuyển input sang nhánh xử lý phù hợp.

What it buys:

- Mỗi nhánh có system prompt và bộ tool riêng, gọn hơn.
- Tránh prompt bloat (không phải nhồi 50 tool vào một prompt).
- Query đơn giản có thể chuyển sang model nhỏ để tiết kiệm.

What it costs:

- Router phân loại sai thì cả nhánh phía sau xử lý sai theo.
- Thêm nhánh mới phải sửa cả router lẫn handler phía sau.

Cơ Chế Evaluator-Optimizer

Hai vai trò tách riêng:

1. **Generator:** tạo output ban đầu từ yêu cầu.
2. **Evaluator (LLM hoặc tool):** chấm output theo tiêu chí rõ ràng, trả về feedback có cấu trúc.
3. **Refinement loop:** generator sửa theo feedback cho tới khi đạt yêu cầu.

What it buys:

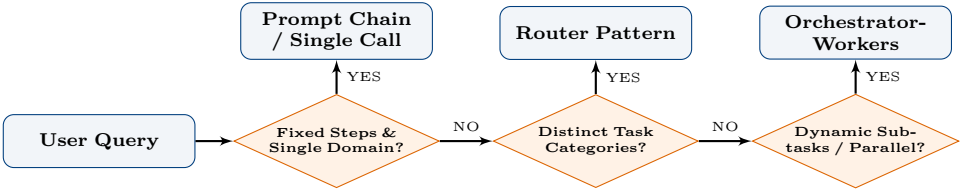
- Chất lượng tăng rõ ở code generation, dịch thuật, soạn văn bản.
- Giống cách con người viết nháp rồi nhờ người khác review.

Chi Phí & Các Failure Mode Thường Gặp

- **Token tăng theo số vòng:** K vòng thì tốn khoảng $K\times$ token, latency cũng tăng theo.
- **Sycophancy:** nếu evaluator dùng chung base model với generator, nó dễ "gật đầu" và bỏ sót những lỗi khó thấy.
- **Diminishing returns:** chất lượng thường tăng nhiều nhất ở vòng đầu; các vòng sau dễ lòng vòng mà không tiến bộ.
- **Cách khắc phục:** dùng **external deterministic tools** (linter, unit test, API checker) làm evaluator thay vì để LLM tự review.

Khi Nào Dùng Workflow, Khi Nào Dùng Agent?

Thuyết trình: Nguyễn Trung Phong | Các câu hỏi cần trả lời trước khi chuyển từ workflow sang agent



Giữ Workflow Đơn Giản Khi:

✓ Input/output schema đã rõ ràng.
✓ Failure mode chặn được bằng regex hoặc Pydantic schema.
✓ Latency SLA chặt (ví dụ dưới 2 giây), token budget hạn chế.

Chỉ Dùng Orchestrator / Agent Khi:

- Không biết trước số lượng và phạm vi subtask.
- Các subtask cần **context isolation**.
- Task chạy lâu, cần failure recovery và **checkpointing**.

Hạn Chế Của Single Long Context

- **Attention Dilution & Context Rot:** Nhồi nhét các ngữ cảnh thô (logs, tools, history) làm tụt dốc tỉ lệ Signal-to-Noise. Mô hình dễ bỏ sót ràng buộc và gặp hiện tượng *Lost in the Middle*.
- **Truncation:** Khi chạm trần ngữ cảnh, cơ chế cắt tỉa bắt buộc diễn ra, dễ làm rụng mất system prompt và kế hoạch ban đầu.
- **Compounding Errors:** Lỗi ở một bước gọi công cụ sẽ bị "neo"vào chuỗi tự hồi quy kế tiếp, khiến agent rơi vào vòng lặp vô tận hoặc trôi khỏi mục tiêu.
- **Sequential Execution & High Cost:** Tiến trình bị khóa trong chuỗi tuần tự $O(N)$; mọi lượt sinh token sau đều chịu chi phí và độ trễ tính toán trên toàn bộ lịch sử thô.

Lợi Thế Của Orchestrator-Workers

- **Context Isolation:** Mỗi subagent sở hữu context window riêng và chỉ nhận đúng công cụ chuyên biệt, loại bỏ rủi ro hallucinated tool arguments.
- **Information Compression:** Worker xử lý hàng chục nghìn tokens thô nhưng chỉ bàn giao bản tóm tắt tinh gọn cho Orchestrator, giữ sạch context cấp cao.
- **Parallel Tooling:** Phân rã bài toán mở rộng theo chiều ngang cho nhiều worker chạy đồng thời, cắt giảm độ trễ thực thi lên tới 90%.
- **Compute Scaling & Model Tiering:** Cho phép tăng token budget hiệu dụng vượt trần đơn agent và phối hợp linh hoạt mô hình lớn với mô hình nhẹ, rẻ.

Mô Hình Orchestrator-Workers (Anthropic)

Thuyết trình: Vũ Việt Hưng | Cơ chế phân rã nhiệm vụ động và tổng hợp song song



Cơ Chế Điều Phối (Workflow Pipeline)

- **Orchestrator:** Tiếp nhận yêu cầu (*In*), phân tích bài toán và quyết định động các subtask cần thực thi.
- **Worker LLMs (LLM Calls 1–3):** Thực thi song song độc lập; mỗi worker chỉ nhận đúng thông tin chuyên biệt.
- **Synthesizer:** Thu thập kết quả từ các worker, giải quyết mâu thuẫn và tổng hợp thành kết quả hoàn chỉnh (*Out*).

Đặc Trưng & Khi Nào Nên Áp Dụng?

- **Dynamic Decomposition:** Số lượng subtask không cố định trước (khác Prompt Chaining hay Parallelization cứng).
- **Context Hygiene:** Loại bỏ attention dilution; worker không thấy toàn bộ history thô, chỉ nhận input cần thiết.
- **Ứng dụng điển hình:** Sửa đổi nhiều file mã nguồn, nghiên cứu tài liệu đa nguồn, kiểm toán pháp lý đa điều khoản.

Anthropic: Lead/Subagent Đáng Giá Đến Đâu?

Thuyết trình: Vũ Việt Hưng | Bài học từ multi-agent research system của Anthropic

Kiến Trúc & Tối Ưu Năng Lực

- **Lead agent:** Giữ research goal, phân rã task, quản lý effort budget và tổng hợp kết quả.
- **Subagents & Parallelism:** Chạy song song (3–5 subagents, 3+ tool calls/lần) → giảm 90% thời gian tìm kiếm.
- **Context hygiene:** Subagents chỉ gửi về kết quả đã cô đọng, hoặc xuất ra Filesystem/Memory để tránh "game of telephone".

Kết quả: Multi-agent (Opus 4 lead, Sonnet 4 subagent) vượt single-agent Opus 4 **90.2%** trên internal eval.

Đánh Đổi & Thách Thức Kỹ Thuật

- **Token Cost:** Agent đơn tốn ≈ 4×, multi-agent tốn ≈ 15× token so với chat (giải thích 80% variance trên BrowseComp).
- **Latency Bottleneck:** Lead agent chờ subagents đồng bộ → 1 subagent bị chậm/treo sẽ chặn toàn bộ pipeline.
- **Compounding Errors:** Lỗi nhỏ tích tụ trong quy trình dài. Cần Checkpoints, Retry logic và Rainbow Deployments để khắc phục.

Kết Luận Theo Đề Cương

Multi-agent đổi token lấy năng lực (chi phí ≈ 15× **so với chat**). Mô hình này chỉ hiệu quả kinh tế cho các tác vụ giá trị cao, có tính song song lớn và vượt quá giới hạn của một context window đơn lẻ.

OpenAI (2025): Tool Loop, Guardrails và Evals

Thuyết trình: Vũ Việt Hưng | Hướng dẫn thực hành: kiểm soát vòng lặp, guardrails và evals

Tool Loop & Mở Rộng Quyền Tự Chủ Từ Từ

- **Vòng lặp do model dẫn dắt:** model sinh tool call → hệ thống chạy tool → observation đưa lại vào context → lặp tới khi gặp điều kiện dừng.
- **Strict schemas:** validate tham số tool call bằng JSON schema/Pydantic trước khi thực thi.
- **Bắt đầu với một agent:** khai thác hết khả năng của một agent rồi mới tách thành nhiều agent.

Guardrails & Evals

- **Guardrails nhiều lớp:** kết hợp LLM-based guardrail, rule-based (regex) và moderation API để lọc input/output.
- **Evals trước tiên:** dựng prototype với model mạnh nhất để có baseline, sau đó mới thử thay bằng model nhỏ hơn.
- **Human intervention:** chuyển cho người khi agent vượt failure threshold hoặc trước các high-risk action (huỷ đơn, hoàn tiền lớn, thanh toán).

Trích Dẫn Từ OpenAI (2025)

"Our general recommendation is to maximize a single agent's capabilities first."
"Think of guardrails as a layered defense mechanism."

LangGraph: Điều Phối Multi-Agent Bằng Cyclic State Graph

Thuyết trình: Vũ Việt Hưng | State schema, reducers và conditional edges

Các Khái Niệm Cốt Lõi Của LangGraph

- **State schema:** một state dictionary có kiểu rõ ràng, được truyền qua mọi node.
- **Reducers:** quy định cách update của từng node được merge vào state (ví dụ: append messages hay ghi đè).
- **Nodes:** mỗi node là một LLM call, một subagent hoặc một deterministic tool.
- **Conditional edges:** hàm Python đọc state để quyết định bước tiếp theo (audit fail → chuyển sang nhánh recovery).

Vì Sao Cần Graph Có Chu Trình?

- **DAG chỉ đi một chiều:** nếu bước retrieval lấy phải văn bản đã bị bãi bỏ, DAG không quay lại được.
- **Cyclic graph cho phép lặp lại:** quay về node planning để tìm văn bản thay thế, kèm retry counter.
- **Bounded loops:** giới hạn số lần retry (và `recursion_limit`) để vòng lặp không chạy mãi.

Giá Trị Cốt Lõi

LangGraph nằm giữa workflow cố định và agent tự do: model vẫn được ra quyết định, nhưng mọi bước chuyển state đều có cấu trúc và **inspect được**.

Durable Execution & Checkpointing: Xử Lý Lỗi Trong Production

Thuyết trình: Vũ Việt Hưng | State persistence, fault recovery và human-in-the-loop

Durable Execution Là Gì?

- Agent chạy task dài (vài phút tới vài giờ) **chắc chắn sẽ gặp sự cố**: rớt mạng, dính rate limit, container bị kill.
- **Không có checkpoint:** lỗi ở bước 9 nghĩa là chạy lại từ bước 1, tốn thêm cả token lẫn thời gian.
- **Checkpointer của LangGraph:** lưu state xuống SQLite/Postgres sau mỗi bước của graph.
- **Fault recovery:** chạy tiếp từ checkpoint hợp lệ gần nhất.

Time Travel & Human-in-the-Loop

- **Interrupts:** graph tạm dừng trước các hành động rủi ro cao (ví dụ: trước khi nộp hồ sơ pháp lý).
- **Human review:** người kiểm tra state, chỉnh sửa nếu cần rồi cho chạy tiếp.
- **Time travel:** fork từ một checkpoint bất kỳ trong quá khứ để thử lại hoặc truy vết lỗi.

Bài Học Theo Đề Cương

Durable execution biến một agent prototype dễ vỡ thành hệ thống có thể đưa vào production, vì phần việc đã làm xong không bị mất khi có sự cố.

Ma Trận Trade-off Giữa Các Pattern

Thuyết trình: Đặng Lâm Tùng (Trưởng nhóm) | Latency, token cost, độ tin cậy và failure mode

Pattern	Control Flow	Token Cost	Latency	Failure Mode Chính	Phù Hợp Nhất Cho
1. Prompt Chaining	Code cố định	$1\times$ (baseline)	Cộng dồn theo số bước	Lỗi cộng dồn, nhưng số bước có giới hạn	Pipeline tuyến tính, trích xuất theo schema cố định.
2. Routing	Code cố định	$\approx 1\times$ ($+1$ lần phân loại)	Thêm 1 bước router	Router phân loại sai	Phân loại ticket hỗ trợ, chọn model hoặc handler phù hợp.
3. Parallelization	Code cố định	$N\times$ (N nhánh)	Bằng nhánh chậm nhất	Nghẽn ở bước gộp kết quả	Voting, chạy song song nhiều guardrail.
4. Evaluator-Optimizer	Vòng lặp có giới hạn	$K\times$ (K vòng)	Tăng theo K	Sycophancy, các vòng sau ít cải thiện	Sinh code, soạn văn bản kỹ thuật.
5. Orchestrator-Workers	Phân cấp (lead/workers)	$\approx$ **15**$\times$ chat*	Chờ worker chậm nhất ($\max t_i$)	Mất thông tin khi tổng hợp	**Research sâu, task chia song song được, cần context isolation.**
6. Autonomous Agent	Model tự quyết định	$\approx 4\times$ chat*, tăng theo số turn	Khó dự đoán	Lỗi cộng dồn, khó kiểm soát	Bài toán mở, debug trong sandbox.

*Số liệu của Anthropic (06/2025), so với một lượt chat thông thường. Các dòng còn lại suy ra từ cấu trúc của pattern.

Simplicity-First (Anthropic, 2024)

"When building applications with LLMs, we recommend finding the simplest solution possible, and only increasing complexity when needed." Bắt đầu từ pattern đơn giản nhất giải quyết được bài toán; chỉ chuyển lên orchestrator hay agent khi evals cho thấy thật sự cần.

Dynamic Orchestration: Kiến Trúc Lead/Subagent Phân Cấp

Thuyết trình: Đặng Lâm Tùng (Trưởng nhóm) | Phân rã task động và context isolation

Kiến Trúc Lead/Subagent Phân Cấp

1. **Lead orchestrator (dynamic planner):** phân tích query ở runtime, tách thành các subgoal độc lập (ví dụ: vốn điều lệ và tỷ lệ sở hữu nước ngoài).
2. **Isolated subagent workers:** tạo worker ngắn hạn cho từng việc (retrieval, audit, drafting), chạy song song, mỗi worker có context riêng.
3. **Typed state handoff:** các agent trao đổi qua schema có kiểu (Pydantic) thay vì văn bản tự do.
4. **Context purging:** scratchpad và search trace của worker bị bỏ đi sau khi đã trích xuất kết quả có cấu trúc gửi về lead.
5. **Centralized checkpointer:** lưu mọi lần chuyển state xuống SQLite/Postgres để có thể khôi phục khi cần.

Vì Sao Context Isolation Hiệu Quả?

- **Context nhỏ và sạch:** thay vì một context khổng lồ chứa mọi thứ, mỗi worker chỉ thấy phần thông tin của mình.
- **Giảm "lost in the middle":** thông tin cần dùng không bị chìm giữa hàng chục nghìn token không liên quan.
- **Fault containment:** kết quả rác hay văn bản hết hiệu lực ở node retrieval bị chặn lại, không lan sang drafter.

Áp Dụng Trong Hệ Thống G4

Trong hệ thống của nhóm, `LegalOrchestrator` chạy song song `LawGraphAgent` (retrieval) và `ClaimAuditorAgent` (audit oracle), sau đó chuyển kết quả cho `DrafterAgent`.

Chọn & Loại Pattern: Vì Sao Không Dùng ReAct Tự Do?

Thuyết trình: Đặng Lâm Tùng (Trưởng nhóm) | Quyết định kiến trúc cho bài toán cần độ tin cậy cao

Pattern Bị Loại: ReAct Loop Không Giới Hạn

- **Bản chất:** một LLM tự gọi tool theo vòng Thought → Action → Observation.
- **3 vấn đề khi chạy production:**
  1. **Lỗi cộng dồn ($P = \prod p_i$):** với $p = 0.92$, 6 bước còn $P \approx 60\%$, 12 bước còn $P \approx 36\%$.
  2. **Token tăng $O(N^2)$:** history dài thêm sau mỗi turn và bị gửi lại toàn bộ.
  3. **Không có fault containment:** state chỉ nằm trong RAM; lỗi mạng hay timeout ở turn cuối là mất hết.

Pattern Được Chọn: Orchestrator-Workers Có Cấu Trúc

- **Bản chất:** bài toán được tổ chức thành state graph; mỗi node chỉ có quyền tự chủ trong giới hạn (bounded autonomy).
- **3 lợi thế:**
  1. **Fault containment:** worker chỉ có quyền hạn hẹp; lỗi bị chặn tại ranh giới của worker.
  2. **Durable execution:** checkpoint sau mỗi bước, có thể retry cục bộ.
  3. **Budget control:** giới hạn token và max turns cho từng node, không có vòng lặp vô hạn.

Bài Học Rút Ra

Unconstrained autonomy là nguồn gốc chính của những lỗi khó đoán. Thiết kế hợp lý là để LLM lo phần phân rã task linh hoạt, còn **control flow và state do code quản lý**.

Fault Tolerance: Deterministic Oracles vs. LLM Self-Critique

Thuyết trình: Đặng Lâm Tùng (Trưởng nhóm) | Tách phần reasoning khỏi phần validation

Giới Hạn Của LLM-as-a-Judge

- **Parametric circularity:** hỏi lại model *"Are you sure?"* thực chất là lấy mẫu lại từ chính phân phối $P_\theta(y|x)$ đã sinh ra lỗi. Không có feedback bên ngoài, self-correction thường không cải thiện được kết quả (Huang et al., 2024).
- **Sycophancy:** evaluator LLM dễ đồng tình với output của generator, nhất là khi dùng chung base model.
- **Không có thông tin real-time:** model không thể tự biết một văn bản luật vừa bị bãi bỏ hôm qua nếu không tra registry bên ngoài.

Validation Bằng Deterministic Oracles

- **Decoupled planes:** tách phần reasoning (mang tính xác suất) khỏi phần validation (tiền định).
- **Binary assertion gates:** dùng linter, type checker, schema validator, registry API... trả về kết quả rõ ràng `VALID` hoặc `INVALID`.
- **Hard circuit breakers:** khi oracle phát hiện vi phạm, graph dừng nhánh hiện tại và chuyển sang nhánh recovery.
- **Audit trail:** lưu kết quả kiểm tra vào durable state để truy vết về sau, cả về kỹ thuật lẫn pháp lý.

Xử Lý Lỗi & Rollback State Trong Graph

Thuyết trình: Đặng Lâm Tùng (Trưởng nhóm) | Xử lý lỗi cục bộ và rollback dựa trên checkpoint

Luồng Thực Thi & Differential Rollback

1. **Checkpoint C_1 (planning):** orchestrator tách query thành Subgoal 1 (điều kiện cấp phép) và Subgoal 2 (tỷ lệ sở hữu nước ngoài) → lưu C_1 (SQLite/Postgres).
2. **Chạy song song:** Worker 1 xử lý Subgoal 1, Worker 2 xử lý Subgoal 2 → lưu C_2.
3. **Oracle phát hiện lỗi:** auditor thấy văn bản dùng cho Subgoal 1 đã hết hiệu lực → kích hoạt conditional failure edge.
4. **Differential rollback:** • Quay state về C_1.
   • **Giữ lại kết quả hợp lệ:** Subgoal 2 vẫn nằm trong state, không phải chạy lại.
   • Chỉ chạy lại Subgoal 1 với văn bản thay thế, retry tối đa $K \leq 2$ lần.
5. **Hoàn tất:** drafter tổng hợp hồ sơ từ dữ liệu đã qua kiểm tra → lưu C_3.

Hệ Thống Đảm Bảo Được Gì?

- **Không cold restart:** không chạy lại các subtask đã hợp lệ, tiết kiệm token và thời gian.
- **Resumable:** rớt mạng hay container sập thì chạy tiếp từ checkpoint gần nhất.
- **Time-travel debugging:** fork từ một checkpoint bất kỳ để tái hiện và chẩn đoán lỗi.

Minh Họa Trong Demo

Trace xử lý việc Nghị định 101/2012 bị bãi bỏ và thay bằng Nghị định 52/2024, chạy trên `LegalOrchestrator` của nhóm.

So Sánh Các Topology: Hub-and-Spoke, Swarm & Cyclic Graph

Thuyết trình: Đặng Lâm Tùng (Trưởng nhóm) | Ba cách tổ chức giao tiếp giữa các agent

1. Hub-and-Spoke (Anthropic)

Mô hình tập trung:

- Lead điều phối; subagent làm việc độc lập rồi báo cáo lại cho lead.
- **Ưu điểm:** lead nắm toàn bộ state; context của các subagent tách biệt.
- **Đánh đổi:** latency phụ thuộc subagent chậm nhất (**straggler effect**, $\max t_i$); tốn nhiều token (khoảng 15× chat).

2. Decentralized Swarm

Peer-to-peer:

- Agent tự handoff và trao đổi trực tiếp với nhau (ví dụ: OpenAI Swarm, nay là OpenAI Agents SDK).
- **Ưu điểm:** linh hoạt với hội thoại nhiều bên, không cần cấu trúc cố định.
- **Rủi ro:** số message có thể tăng $O(N^2)$; dễ kẹt vòng lặp ping-pong; khó rollback state.

3. Cyclic State Graph

LangGraph:

- Điều phối qua state graph với typed schema và conditional edges.
- **Ưu điểm:** kiểm soát được vòng lặp; có sẵn checkpointing và time travel.
- **Đánh đổi:** phải thiết kế state schema cẩn thận từ đầu; kém linh hoạt hơn swarm.

Lựa Chọn Cho Hệ Thống Production

Hệ thống cần độ tin cậy cao nên tránh swarm không có ràng buộc. Hướng phổ biến hiện nay là **hub-and-spoke để phân công việc**, kết hợp với **cyclic state graph để xử lý lỗi và recovery**.

Nguyên Tắc Thiết Kế Agent Cho Production

Thuyết trình: Đặng Lâm Tùng (Trưởng nhóm) | Tổng hợp bài học từ Anthropic, OpenAI và LangGraph

1. Simplicity-First

- Mặc định dùng prompt chaining và deterministic routing.
- Code lo control flow, LLM chỉ lo các bước reasoning.
- Workflow cố định đã đủ thì không thêm dynamic orchestration.

2. Context Isolation

- Khi thật sự cần multi-agent, dùng mô hình lead/subagent.
- Không dùng chung một scratchpad cho nhiều agent.
- Token tăng (khoảng 15× chat) là cái giá của context sạch, chỉ đáng khi task đủ giá trị.

3. Durable Execution

- Checkpoint mọi lần chuyển state.
- Validation bằng external deterministic oracles.
- Đặt giới hạn retry (circuit breaker) cho mọi vòng lặp.

Thông Điệp Chính

Kiến trúc agent tốt nhất là kiến trúc đơn giản nhất giải quyết được bài toán một cách tin cậy. **Độ phức tạp là chi phí, không phải tính năng.**

Tổng Kết & Thảo Luận

Các kết luận chính của Seminar S1-7

Thông Điệp Chính:

Agent linh hoạt nhưng dễ vỡ; với task đã rõ các bước, workflow ổn định và rẻ hơn.
Khi bài toán buộc phải điều phối động, **lead/subagent với context isolation** kết hợp **checkpointing của LangGraph** là hướng đi bền vững nhất để đưa vào production.

Cảm Ơn Thầy Và Các Bạn Đã Lắng Nghe!

Nhóm G4 rất mong nhận được câu hỏi và góp ý từ TS. Lê Xuân Bách và các bạn.

Nhóm G4: Đặng Lâm Tùng (Trưởng nhóm) · Nguyễn Trung Phong · Vũ Việt Hưng | `CO5151 -- HK261`